ЛЕКЦИЯ 1.5 · LLMOps

Kubernetes

Оркестрация контейнеров: балансировка, масштабирование и отказоустойчивость приложений

Чернов Евгений

Telegram: @awe_eu

ГДЕ МЫ НАХОДИМСЯ

Программа курса: элементы промышленной разработки для ML-инженера

БЛОК 1

Элементы промышленной разработки

- 1.1 LLM-проект: этапы и инструменты
- 1.2 Сети, модели OSI и TCP/IP
- 1.3 API на L7: HTTP и gRPC
- 1.4 Контейнеризация и виртуализация
- 1.5 **Основы Kubernetes**

БЛОК 2

Обучение и инференс LLM на GPU

- 2.1 Архитектура GPU
- 2.2 Эффективная утилизация GPU
- 2.3 Фреймворки деплоя LLM
- 2.4 Разработка realtime-сервиса

ПОСТАНОВКА ЗАДАЧИ

Зачем нужен Kubernetes?

Представьте систему в проде, которой нужно постоянно обеспечивать четыре свойства:

1

Балансировка

Равномерно распределять нагрузку по нодам и репликам

2

Обновления

Выкатывать новые версии без простоя и с откатом

3

Отказоустойчивость

Перезапускать упавшие сервисы и держать нужное число реплик

4

Изоляция

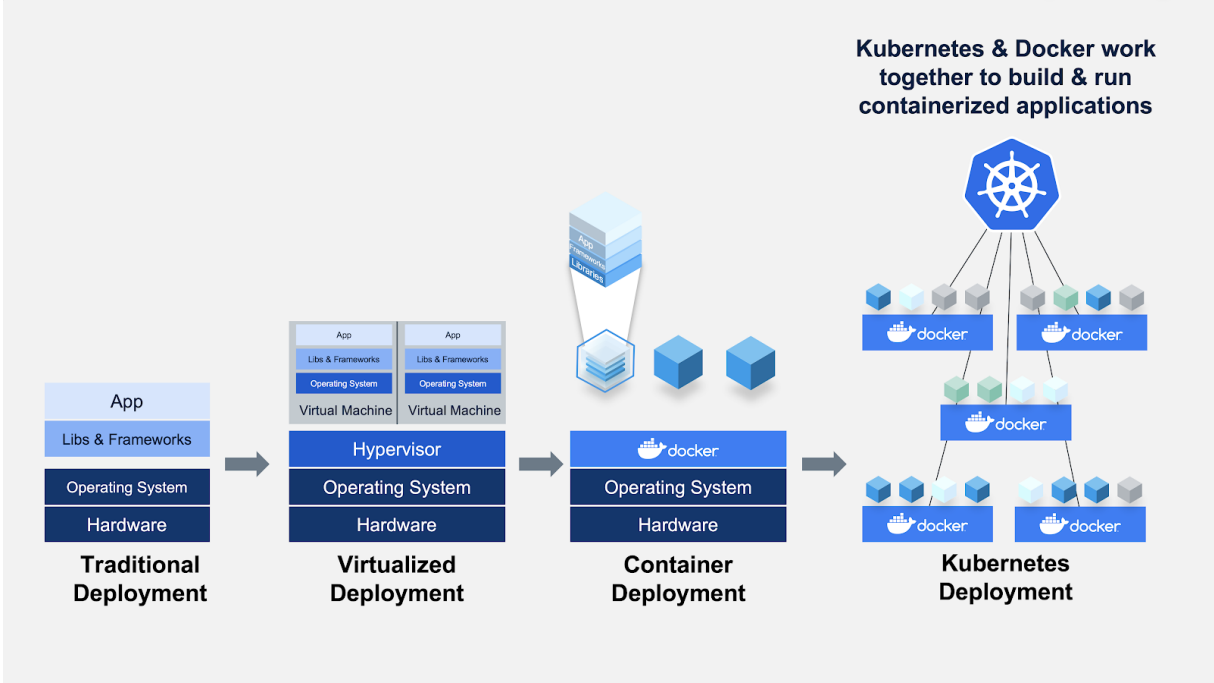
Разделять окружения и ресурсы между приложениями

!

А что умеет Docker? **Только изоляцию.** Балансировку, обновления и самовосстановление берёт на себя Kubernetes.

КОНТЕКСТ

От «железа» к оркестрации контейнеров



Traditional → Virtualized → Container → Kubernetes. Контейнеры дают лёгкую изоляцию поверх одной ОС, а Kubernetes добавляет управление их жизненным циклом на кластере машин.

ОПРЕДЕЛЕНИЕ

Что такое Kubernetes

Kubernetes (k8s) — система для автоматизации деплоя, масштабирования и управления контейнеризированными приложениями на кластере машин.

Поддерживаемые container runtime

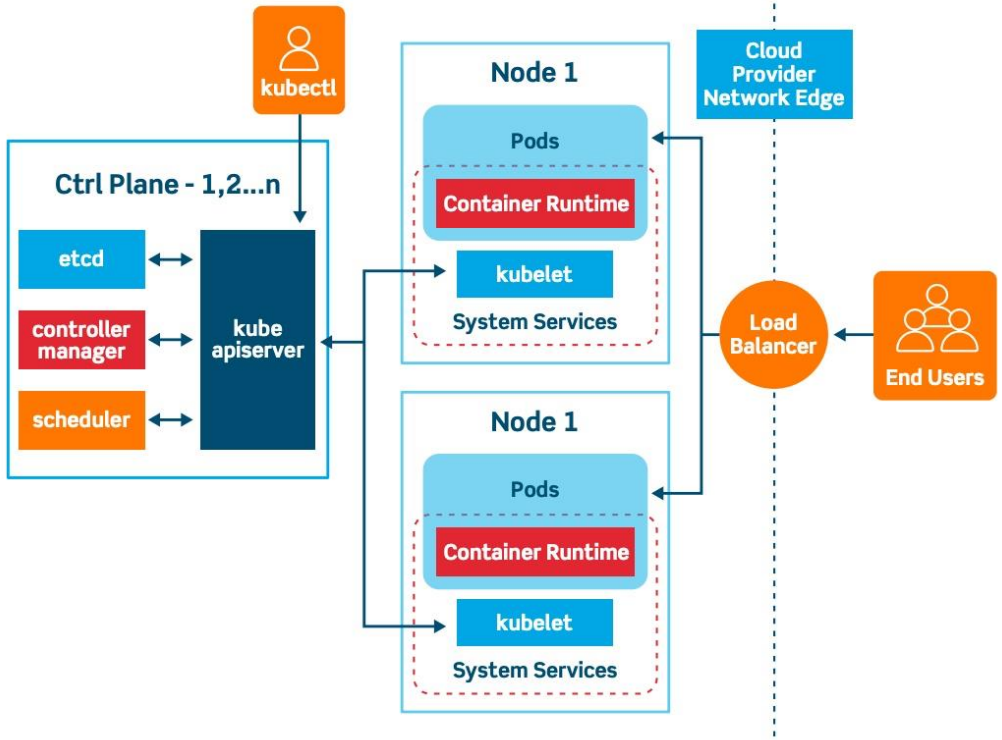
- **containerd** — стандарт де-факто, используется в k8s
- **CRI-O** — лёгкий runtime под Kubernetes CRI
- **Docker** — через shim; исторически самый известный
- **podman / rkt** — альтернативные движки контейнеров

Как запустить кластер

1. **Локально**
   minikube — учебный кластер на своём ноутбуке
2. **Лёгкий дистрибутив**
   k3s — минимальный k8s для edge и стендов
3. **В облаке**
   Yandex Cloud через terraform или меню администратора

АРХИТЕКТУРА

Как устроен кластер: Control Plane и рабочие ноды



Два уровня

- **Control Plane**
  «мозг» кластера: принимает решения, хранит состояние, планирует размещение подов
- **Worker Nodes**
  рабочие машины: на них реально запускаются контейнеры с приложениями
- **kubectl**
  CLI администратора — команды идут в kube-apiserver

КОМПОНЕНТЫ

Из чего собран кластер

CONTROL PLANE — управляющий слой

kube-apiserver

единая точка входа в кластер; через него идут все команды и запросы

etcd

распределённое хранилище состояния: что и где запущено

scheduler

решает, на какую ноду поставить новый под

controller-manager

следит, чтобы фактическое состояние сходилось с желаемым

WORKER NODE — рабочий слой

kubelet

агент ноды: запускает контейнеры и следит за их состоянием

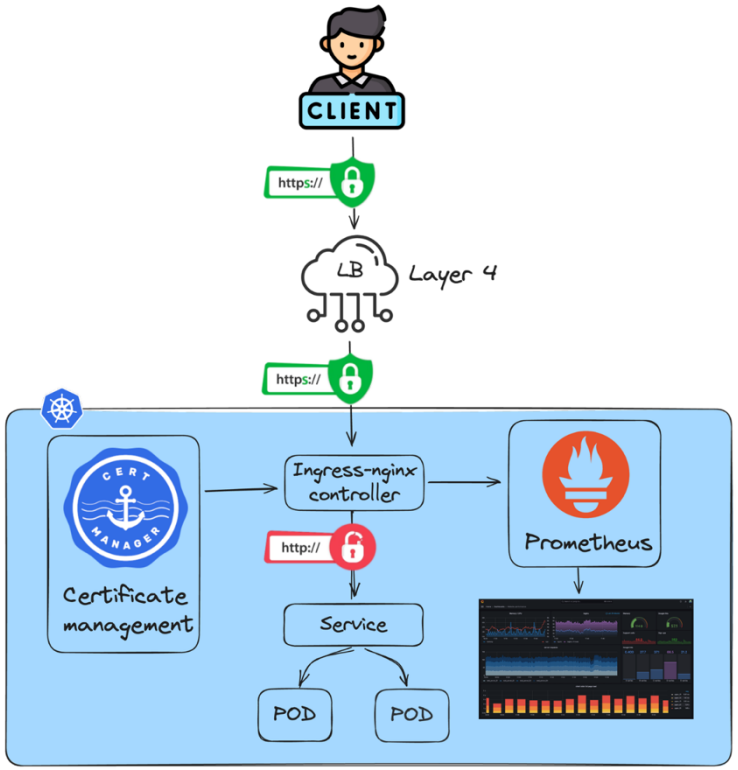
container runtime

движок контейнеров (containerd / CRI-O), исполняет образы

kube-proxy

сетевые правила: маршрутизация трафика к подам сервиса

Как клиент и админ достигают сервисов в кластере



Клиент → снаружи
Трафик идёт через Load Balancer (L4) на Ingress-контроллер, а тот — на нужный Service

Ingress
точка входа HTTP/HTTPS; терминирует TLS и роутит по хостам и путям

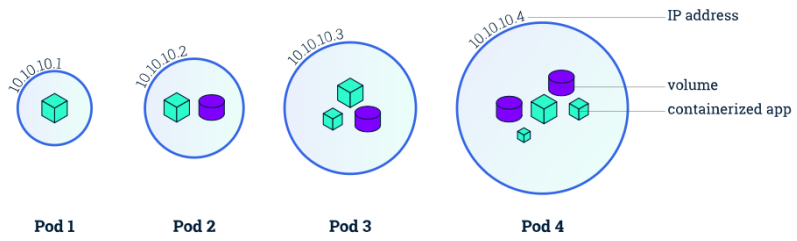
Админ → kubectl
команды идут в kube-apiserver, а не напрямую к подам

Мониторинг
Prometheus собирает метрики, Grafana их визуализирует

РЕСУРС · POD

Pod — атомарная единица запуска

Pod — наименьшая сущность в k8s. Это один или несколько контейнеров с общими сетью, IP-адресом и томами.



Каждый Pod получает собственный IP; контейнеры внутри общаются через localhost.

pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: hse-pod-1
  labels:
    app: frontend
spec:
  containers:
  - name: hse-pod-1
    image: aesedeu/nginx-hse-ai:1.0
    resources:
      requests:  { cpu: 250m, memory: 64Mi }
      limits:    { cpu: 500m, memory: 128Mi }
    ports:
      - containerPort: 80
```

РЕСУРС · REPLICASET

ReplicaSet — поддержание нужного числа реплик

replicaset.yaml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: hse-rs
spec:
  replicas: 4
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels: { app: frontend }
    spec:
      containers:
      - name: frontend
        image: httpd:latest
```

Что делает ReplicaSet

1. **Держит N подов**
   Всегда поддерживает заданное число реплик (replicas: 4)

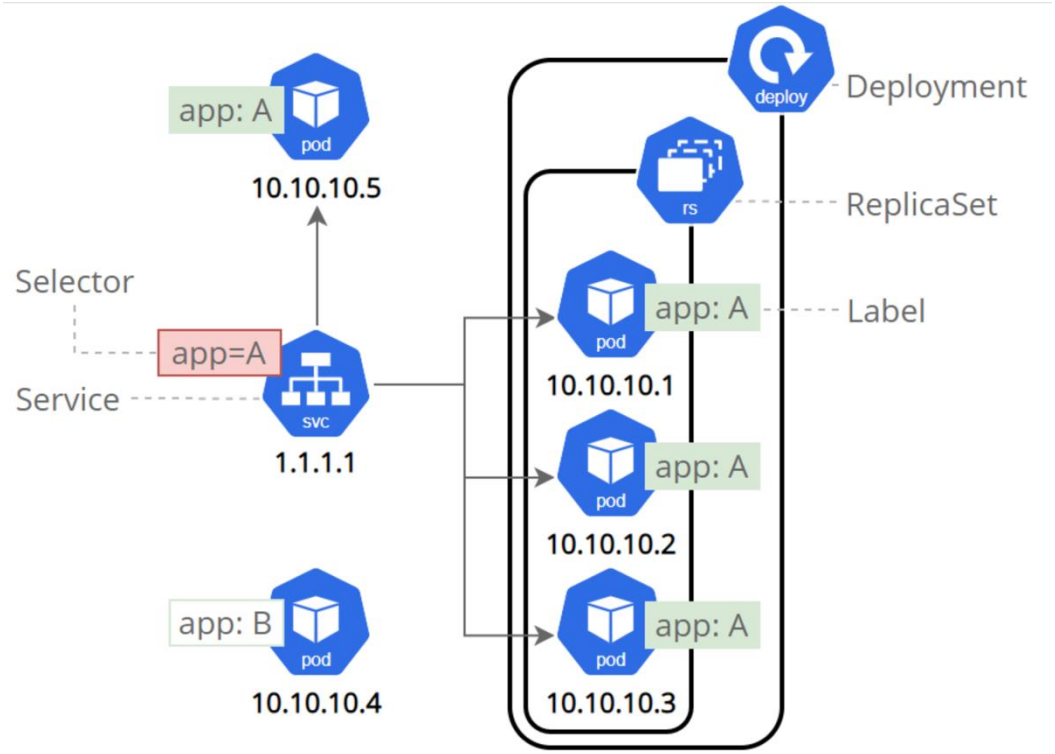
2. **Подбор по меткам**
   Управляет подами, чьи labels совпадают с selector

3. **Самовосстановление**
   Удалили под — RS тут же поднимает новый на замену

4. **Масштабирование**
   Меняем replicas в манифесте — число подов подстраивается

РЕСУРС · DEPLOYMENT

Deployment — управление состоянием приложения



Deployment → ReplicaSet → Pods

- **Декларативные обновления**
  задаём желаемый образ — k8s сам плавно заменяет поды
- **История ревизий**
  kubectl rollout history — видно все версии деплоя
- **Откат в один шаг**
  kubectl rollout undo возвращает предыдущую ревизию
- **RollingUpdate**
  maxUnavailable / maxSurge — обновление без простоя

РЕСУРС · DEPLOYMENT

Стратегия обновления и операции rollout

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: hse-deployment }
spec:
  replicas: 4
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  selector:
    matchLabels: { app: backend }
  template:
    metadata: { labels: { app: backend } }
    spec:
      # nodeSelector: { load: high }
      containers:
      - { name: web, image: nginx:latest }
```

rollout — управление выкаткой

```
$ kubectl rollout history deployment/hse-deployment
$ kubectl set image deploy/hse web=nginx:1.27
$ kubectl rollout status deployment/hse
$ kubectl rollout undo deployment hse   # откат
```

Размещение по нодам (nodeSelector)

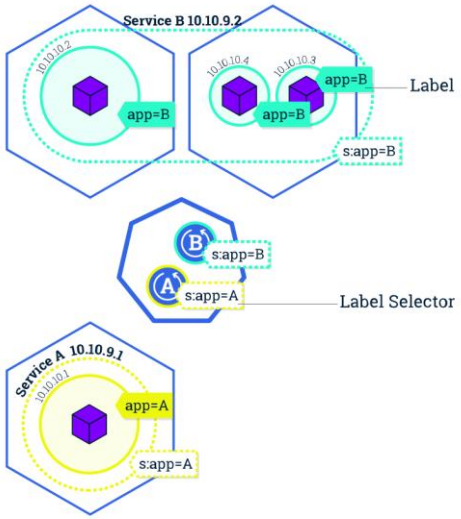
`kubectl label nodes minikube-m02 load=high`

Добавляем nodeSelector в манифест — и поды переезжают только на ноды с нужной меткой.

РЕСУРС · SERVICE

Service — стабильная точка доступа к подам

Service — логическая абстракция: единый IP и DNS-имя, за которыми через **label-селекторы** скрывается набор подов. Поды приходят и уходят — адрес Service остаётся постоянным.



Как работают селекторы

Под
несёт метки, например app: frontend.

Service
задаёт selector app: frontend и связывает все совпавшие поды.

Трафик
балансируется между ними автоматически.

РЕСУРС · SERVICE

Три типа Service

1

ClusterIP

ПО УМОЛЧАНИЮ

Внутренний IP, доступный только внутри кластера. Для связи сервисов между собой.

2

NodePort

ПОРТ НА НОДАХ

Открывает один порт на каждой ноде (30000–32767). Простой внешний доступ для стендов.

3

LoadBalancer

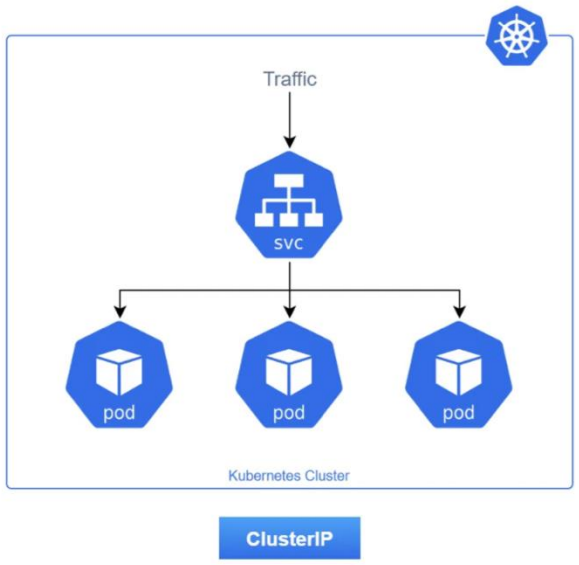
ТОЛЬКО В ОБЛАКЕ

Провайдер поднимает внешний балансировщик с публичным IP перед сервисом.

РЕСУРС · SERVICE

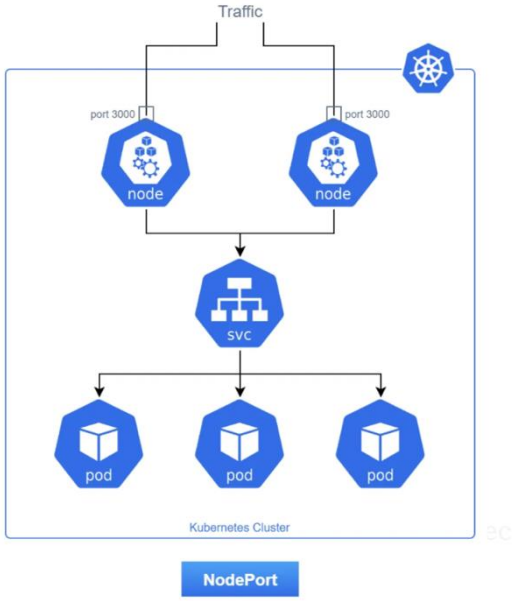
Три типа Service — наглядно

ClusterIP



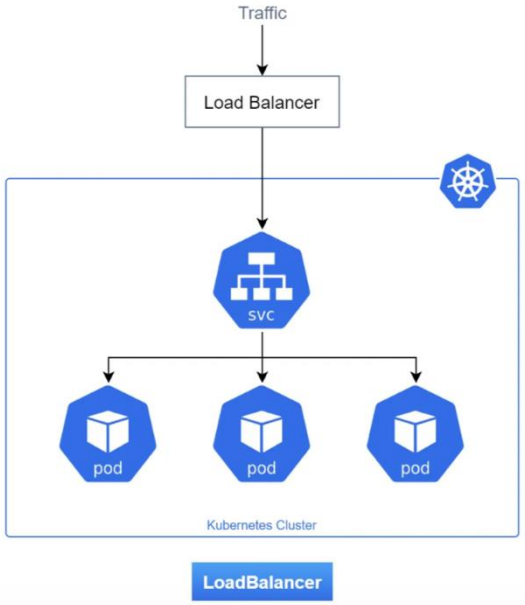
доступ внутри кластера

NodePort



порт на каждой ноде

LoadBalancer

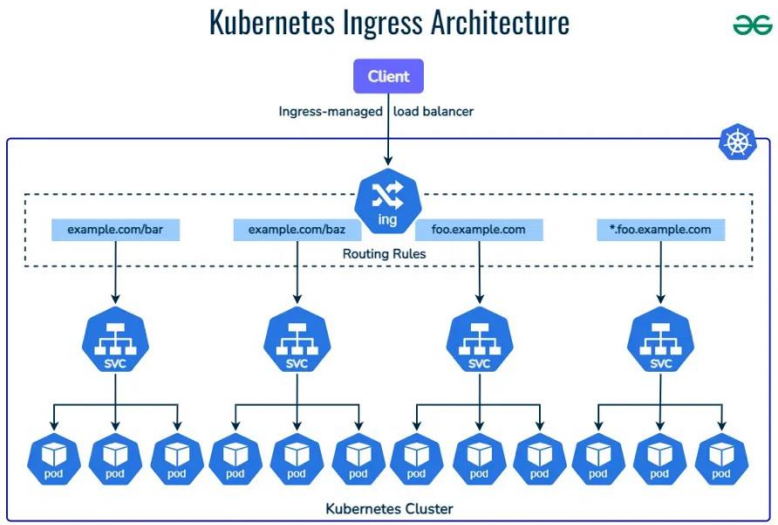


внешний IP из облака

РЕСУРС · INGRESS

Ingress — единая точка входа HTTP/HTTPS

Ingress — объект k8s API, который выставляет HTTP/HTTPS-роуты снаружи кластера и по хосту/пути направляет трафик на внутренние сервисы. Работает через Ingress-контроллер (например, nginx).



Зачем нужен

- Один вход вместо множества NodePort / LoadBalancer
- Роутинг по хостам: hse-ai.ru, backend.hse-ai.ru
- Роутинг по путям: /, /api, /metrics
- Терминация TLS в одном месте

КОНФИГУРАЦИЯ

ConfigMap и Secrets — отделяем настройки от кода

ConfigMap — несекретные настройки

configmap.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata: { name: my-config }
data:
  config.yaml: |
    DATA_1: 100
    DATA_2: 200
```

Монтируется как файл: `volumeMounts → /etc/config.`
Внутри пода конфиг доступен по пути как обычный YAML-файл.

Secret — чувствительные данные

secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata: { name: my-secret }
type: Opaque
data:
  username: YWRtaW4=    # admin   (base64)
  password: c2VjcmV0    # secret  (base64)
```

Значения в base64, не шифрование: `echo -n 'admin' | base64`
Пробрасывается в под как переменные окружения через secretKeyRef.

НАБЛЮДАЕМОСТЬ

Мониторинг кластера: Prometheus + Grafana

1. **kube-prometheus-stack**
   Ставим Helm-чартом в namespace monitoring — сразу Prometheus, Grafana и Alertmanager

2. **Node Exporter**
   Автоматически встаёт на все ноды и отдаёт метрики хостов

3. **Grafana ↔ Prometheus**
   Grafana подключается к prometheus-operated:9090; дашборды кластера из коробки

4. **ServiceMonitor**
   Кастомный сбор метрик приложения (напр. vllm-backend /metrics)

установка стека

```
$ kubectl create namespace monitoring
$ helm install prometheus \
    prometheus-community/kube-prometheus-stack \
    --namespace monitoring \
    --set grafana.service.type=LoadBalancer \
    --set grafana.adminPassword=admin
```

Доступ через Ingress

grafana.hse-ai.ru
prometheus.hse-ai.ru
логин / пароль Grafana: admin / admin

ПРАКТИКА

Поднимаем учебный кластер minikube

cluster setup

```
# 3-нодовый кластер (на macOS — драйвер qemu)
$ minikube start --driver qemu --nodes 3 \
    --cpus 3 --memory 8G --disk-size 80G

# включаем аддоны
$ minikube addons enable ingress
$ minikube addons enable metrics-server
$ minikube addons enable dashboard

# через Makefile проекта
$ make setup          # поднять кластер
$ make update_hosts   # прописать hse-ai.ru

# сборка и публикация образа
$ docker buildx build --platform \
    linux/amd64,linux/arm64 \
    -t aesedeu/nginx-hse-ai:1.0 --push .
```

Что происходит при setup

- **3 ноды** — один minikube эмулирует полноценный многонодовый кластер
- **ingress** — поднимается nginx Ingress-контроллер для внешнего доступа
- **metrics-server** — источник метрик CPU/RAM для kubectl top и автоскейла
- **/etc/hosts** — домены *.hse-ai.ru маппятся на IP minikube (minikube tunnel)

ПРАКТИКА

Сценарий занятия: от пода к мониторингу

Part 1	**Pods · Service · Ingress**	Создаём поды, ходим между ними по IP из ubuntu-пода, поднимаем Service и Ingress, затем ReplicaSet — и видим самовосстановление подов
Part 2	**Deployment**	Rolling-обновление образа, история ревизий и откат; метки нод и nodeSelector — переносим поды на нужную ноду
Part 3	**ConfigMap · Secrets**	Монтируем конфиг в /etc/config, пробрасываем секреты в переменные окружения, проверяем балансировку по именам подов
Part 4	**Monitoring**	Ставим kube-prometheus-stack, смотрим дашборды в Grafana, добавляем ServiceMonitor для метрик приложения

МАНИФЕСТЫ? МАНИФЕСТЫ!

Live demo

Разбираем YAML-манифесты и применяем их в кластере, шаг за шагом

- `kubectl apply -f .`
- `kubectl get pods -o wide`
- `minikube tunnel`
- `kubectl rollout undo …`

ИТОГИ

Ключевые абстракции Kubernetes

Pod

атомарная единица запуска: контейнеры с общими сетью и томами

ReplicaSet

держит заданное число одинаковых подов, восстанавливает упавшие

Deployment

декларативные обновления, история ревизий и откат

Service

стабильный IP/DNS и балансировка по label-селекторам

Ingress

единый HTTP/HTTPS-вход и роутинг по хостам и путям

ConfigMap / Secret

конфигурация и секреты отдельно от образа